

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №4
по курсу «Логика и основы алгоритмизации в инженерных задачах»
на тему «Бинарное дерево поиска»

Выполнили:
студенты групп 24ВВВЗ

Масюк А. Р.
Мамонтов Н. П.

Приняли:
Юрова О. В.
Деев М. В.

Пенза 2025

Цель работы

Освоить на практике принципы работы с бинарными деревьями поиска на языке C. Закрепить навыки реализации базовых операций над деревьями: добавление элементов, визуализация, поиск и подсчёт вхождений. Изучить способы обработки рекурсивных структур данных и оценить сложность алгоритмов, связанных с деревьями.

Лабораторное задание

1. Реализовать алгоритм поиска вводимого с клавиатуры значения в уже созданном дереве.
2. Реализовать функцию подсчёта числа вхождений заданного элемента в дерево.
3. * Изменить функцию добавления элементов для исключения добавления одинаковых символов.
4. * Оценить сложность процедуры поиска по значению в бинарном дереве.

Работа программы

```
ЛАБОРАТОРНАЯ РАБОТА №4
-1 - окончание построения дерева
Введите число: 10
Введите число: 1
Введите число: 12
Введите число: 13
Введите число: 8
Введите число: 7
Введите число: 8
Введите число: 9
Введите число: 3
Введите число: 4
Введите число: -1
Построение дерева окончено
Дерево:
  1
   3 4
    7 8
     8 9
      10
       12
        13
```

Построение дерева

```
Дерево:
  1
   3 4
    7 8
     8 9
      10
       12
        13

Введите число для поиска: 4
Элемент 4 найден в дереве.
Число вхождений элемента 4 в дереве: 1
```

Реализация поиска числа и подсчёта количества вхождений элемента

Вывод

В ходе выполнения лабораторной работы были освоены методы работы с бинарными деревьями поиска на языке C. Реализованы функции добавления элементов, визуализации дерева, поиска элемента и подсчёта числа его вхождений. Изучены особенности рекурсивной обработки древовидных структур. Проведён анализ сложности алгоритма поиска в бинарном дереве. Написана программа, позволяющая наглядно продемонстрировать работу с деревьями и выполнить основные операции над ними.

Листинг

```
#include <stdio.h>
#include <stdlib.h>
#include <locale.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* root = NULL;

struct Node* CreateTree(struct Node* r, int data)
{
    if (r == NULL)
```

```

{
    r = (struct Node*)malloc(sizeof(struct Node));
    if (r == NULL)
    {
        printf("Ошибка выделения памяти");
        exit(0);
    }

    r->left = NULL;
    r->right = NULL;
    r->data = data;
    return r;
}

if (data > r->data)
    r->right = CreateTree(r->right, data);
else
    r->left = CreateTree(r->left, data);

return r;
}

struct Node* search(struct Node* r, int key)
{
    if (r == NULL || r->data == key)
        return r;

    if (r->data < key)
        return search(r->right, key);

    return search(r->left, key);
}

int countOccurrences(struct Node* r, int key)
{

```

```

        if (r == NULL)
            return 0;

        int count = 0;
        if (r->data == key)
            count = 1;

        return count + countOccurrences(r->left, key) + countOccurrences(r->right, key);
    }

```

```

void print_tree(struct Node* r, int l)

```

```

{
    if (r == NULL)
    {
        return;
    }

    print_tree(r->left, l + 1);

    for (int i = 0; i < l; i++)
    {
        printf("    ");
    }
    printf("%d\n", r->data);

    print_tree(r->right, l + 1);
}

```

```

void free_tree(struct Node* r)

```

```

{
    if (r == NULL) return;
    free_tree(r->left);
    free_tree(r->right);
    free(r);
}

```

```

int main()
{
    setlocale(LC_ALL, "");
    int D, start = 1;

    root = NULL;
    printf("ЛАБОРАТОРНАЯ РАБОТА №4\n\n");
    printf("-1 - окончание построения дерева\n\n");

    while (start)
    {
        printf("Введите число: ");
        scanf_s("%d", &D);
        if (D == -1)
        {
            printf("Построение дерева окончено\n\n");
            start = 0;
        }
        else
            root = CreateTree(root, D);
    }

    printf("Дерево:\n");
    print_tree(root, 0);
    printf("\n");

    printf("Введите число для поиска: ");
    scanf_s("%d", &D);

    struct Node* result = search(root, D);
    if (result != NULL)
        printf("Элемент %d найден в дереве.\n", D);
    else
        printf("Элемент %d не найден в дереве.\n", D);

    int occurrences = countOccurrences(root, D);
    printf("Число вхождений элемента %d в дереве: %d\n", D, occurrences);
}

```

```
free_tree(root);
```

```
return 0;
```

```
}
```